



SGCI

Backend — Documentação Técnica Aprofundada

API REST em Quarkus · Java 21 · Análise do código-fonte

Quarkus 3.37.1

Hibernate ORM + Panache

PostgreSQL + Flyway

SmallRye JWT

Gerado em 06/07/2026

Sumário

1. Panorama e princípios de design
2. Configuração da aplicação (`application.properties`)
3. Inicialização e sementeira (`DataSeeder`)
4. Camada de entidades (Panache Active Record)
5. Enums de domínio
6. Camada de repositórios (DAO)
7. Camada de serviços (BO) — regras de negócio comentadas
8. Camada de controllers (JAX-RS)
9. DTOs e validação (Bean Validation)
10. Segurança — JWT em cookie, passo a passo
11. Auditoria automática por filtro de resposta
12. Tratamento de exceções
13. Migrações Flyway (V1–V8)
14. Ciclo de vida completo de uma crise (walkthrough)

1. Panorama e princípios de design

O backend do SGCI é uma API REST stateless construída em **Quarkus 3.37.1 sobre Java 21**. Ele concentra toda a lógica de negócio de gestão de crises e persiste os dados em PostgreSQL. O cliente Angular apenas consome os endpoints JSON.

Arquitetura em camadas

HTTP →	Controller (JAX-RS)	valida DTO (@Valid) e autorização (@RolesAllowed)
	Service (BO)	regra de negócio, @Transactional
	Repository (DAO/Panache)	queries e acesso a dados
	Entity (Hibernate ORM)	mapeamento objeto ↔ tabela
	PostgreSQL	schema versionado por Flyway

Os comentários do próprio código nomeiam o Service como **BO (Business Object)** e o Repository como **DAO**, deixando a separação de responsabilidades explícita.

Cinco decisões de design que definem o projeto

#	Decisão	Consequência prática
1	Panache Active Record : entidades com campos <code>public</code> , sem getters/setters.	Código enxuto; acesso direto a <code>crise.status</code> .
2	Sem associações JPA : relações são colunas <code>Long</code> (ex.: <code>tipoCriseId</code>), não <code>@ManyToOne</code> .	Integridade referencial fica só no banco (FK do Flyway); nenhum <code>cascade/lazy-loading</code> .
3	DTOs como record : <code>*Request</code> (entrada validada) e <code>*Response</code> (saída via <code>from()</code>).	Entidades nunca vazam para a API; a senha nunca é serializada.
4	Autenticação por JWT em cookie HttpOnly (não no header Authorization).	Token inacessível ao JS → proteção contra XSS.
5	Soft-delete : DELETE marca <code>active=false</code> .	Histórico preservado; nada é apagado fisicamente.

Estrutura de pacotes

```
br.com.crisismanagement
├─ config/           DataSeeder                (bootstrap de dados)
├─ controllers/      11 resources REST
├─ dto/              records Request / Response
├─ entities/         9 entidades + enums/ (4)
├─ exceptions/       InvalidCredentialsException + Mapper
├─ repositories/     10 DAOs Panache
├─ security/         AuthCookieFactory + AuditoriaLogFilter
└─ services/         11 BOs
```

2. Configuração da aplicação

Todo o comportamento de infraestrutura é declarado em `src/main/resources/application.properties` :

```
# App + servidor HTTP
quarkus.application.name=crisis-management-api
quarkus.http.port=8080

# Datasource PostgreSQL (porta 5433)
quarkus.datasource.db-kind=postgresql
quarkus.datasource.username=postgres
quarkus.datasource.password=postgres
quarkus.datasource.jdbc.url=jdbc:postgresql://localhost:5433/crisis_management

# Schema é do Flyway; Hibernate apenas valida (não altera tabelas)
quarkus.hibernate-orm.database.generation=validate
quarkus.flyway.migrate-at-start=true
quarkus.flyway.baseline-on-migrate=true

# JWT assinado (RS256) lido de COOKIE, não do header Authorization
mp.jwt.verify.issuer=https://crisis-management/issuer
mp.jwt.verify.publickey.location=publicKey.pem
smallrye.jwt.sign.key.location=privateKey.pem
mp.jwt.token.header=Cookie
mp.jwt.token.cookie=jwt

# CORS para o dev server Angular (credenciais on = cookie cruza origens)
quarkus.http.cors.enabled=true
quarkus.http.cors.origins=http://localhost:4200
quarkus.http.cors.access-control-allow-credentials=true
```

Combinação-chave: `database.generation=validate + migrate-at-start=true` significa que o Flyway é a única fonte de verdade do schema, e o Hibernate apenas confere, no start, que as entidades batem com as tabelas. Se divergirem, a aplicação nem sobe — um mecanismo de segurança.

3. Inicialização e sementeira — DataSeeder

Ao subir, o Quarkus dispara um `StartupEvent`. O `DataSeeder` o observa e cria, de forma **idempotente** (só se ainda não existirem), dois usuários iniciais com senha hashada em BCrypt.

```

@ApplicationScoped
public class DataSeeder {
    @Transactional
    void onStart(@Observes StartupEvent event) {
        seedAdmin();
        seedComum();
    }
    private void seedAdmin() {
        if (userRepository.findByEmail("admin@crisis.local").isPresent()) return;
        User admin = new User();
        admin.name = "Administrador";
        admin.email = "admin@crisis.local";
        admin.password = BcryptUtil.bcryptHash("Admin@123"); // nunca em texto puro
        admin.perfil = PerfilUsuario.ADMIN;
        admin.active = true;
        admin.createdAt = LocalDateTime.now();
        userRepository.persist(admin);
    }
    // seedComum(): usuario@crisis.local / Comum@123, perfil COMUM,
    // vinculado ao 1º departamento existente (ou null).
}

```

Perfil	E-mail	Senha	Departamento
ADMIN	admin@crisis.local	Admin@123	—
COMUM	usuario@crisis.local	Comum@123	1º cadastrado (CTI)

4. Camada de entidades (Panache Active Record)

Todas estendem `PanacheEntityBase`, têm id `@GeneratedValue(IDENTITY)` e enums gravados como texto (`@Enumerated(EnumType.STRING)`). Repare que as FKs são `Long` puros:

```
@Entity @Table(name = "crises")
public class Crise extends PanacheEntityBase {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    public Long id;
    @Column(nullable = false, length = 200) public String titulo;
    @Column(nullable = false, length = 1000) public String descricao;
    @Enumerated(EnumType.STRING) public StatusCrise status;
    @Enumerated(EnumType.STRING) public NivelCriticidade nivel;
    public Long tipoCriseId; public Long campusId; // FK por id
    public Long cenarioId; public Long departamentoId; // (não @ManyToOne)
    public boolean active; public LocalDateTime createdAt;
}
```

Catálogo de entidades

Entidade / Tabela	Campos
User · users	id, name, email (único), password (BCrypt), perfil, departamentold?, active, createdAt
Crise · crises	id, titulo, descricao, status, nivel, tipoCrisId, campusId, cenariold?, departamentold, active, createdAt
AcaoCrise · acoes_crise	id, descricao, tipo, crisId, usuariold, createdAt
RelatorioCrise · relatorios_crise	id, titulo, resumoOcorrido, impactosIdentificados?, acoesRealizadas?, resultadoFinal?, recomendacoes?, crisId (único), usuariold, createdAt
Instituicao · instituicoes	id, nome, sigla, active, createdAt
Campus · campi	id, nome, instituicaold, active, createdAt
Departamento · departamentos	id, nome, sigla, active, createdAt
TipoCrise · tipos_crise	id, nome, descricao?, active, createdAt
CenarioCrise · cenarios_crise	id, descricao, nivelSugerido?, tipoCrisId, departamentold, active, createdAt
AuditoriaLog · auditoria_logs	id, usuariold, usuarioNome, acao, metodoHttp, entidadeAfetada, recurso, statusResposta, dataHora

(? = coluna anulável)

5. Enums de domínio

Enum	Valores	Papel
NivelCriticidade	BAIXO · MEDIO · ALTO · CRITICO	Gravidade da crise
StatusCrise	ABERTA · EM_ANDAMENTO · RESOLVIDA · ENCERRADA	Ciclo de vida da crise
TipoAcao	CONTENCAO · COMUNICACAO · MONITORAMENTO · RESOLUCAO	Categoria de cada ação registrada
PerfilUsuario	ADMIN · COMUM	Papel de acesso — o nome vira a <i>role</i> do JWT

O comentário no código de `PerfilUsuario` é explícito: "O nome do enum é usado como 'role' (claim *groups* do JWT) para as verificações de `@RolesAllowed` ." Ou seja, adicionar um perfil novo aqui já o torna utilizável nas anotações de segurança.

6. Camada de repositórios (DAO)

Cada repositório implementa `PanacheRepository<T>`, herdando `persist`, `findByIdOptional`, `count`, `list` etc. Só os métodos específicos de consulta são escritos.

USERREPOSITORY — BUSCA POR E-MAIL (USADA NO LOGIN E NA UNICIDADE)

```
public Optional<User> findByEmail(String email) {
    return find("email", email).firstResultOptional();
}
public List<User> listAllOrdered() { return list("order by name"); }
```

CRISEREPOSITORY — CONTAGENS PARA O DASHBOARD

```
public List<Crise> listAllOrdered() { return list("order by createdAt desc"); }
public long countByStatus(StatusCrise status) { return count("status = ?1", status); }
public long countByNivel(NivelCriticidade nivel) { return count("nivel = ?1", nivel); }
```

AUDITORIAREPOSITORY — FILTRO DINÂMICO

O WHERE é montado conforme os parâmetros preenchidos (nulos/vazios são ignorados):

```
public List<AuditoriaLog> search(String entidade, String acao, String usuario) {
    StringBuilder query = new StringBuilder("1 = 1");
    Map<String, Object> params = new HashMap<>();
    if (entidade != null && !entidade.isBlank()) {
        query.append(" and entidadeAfetada = :entidade"); params.put("entidade", entidade);
    }
    // idem para acao (=) e usuario: lower(usuarioNome) like %...%
    query.append(" order by dataHora desc");
    return find(query.toString(), params).list();
}
```

MÉTODOS POR REPOSITÓRIO

Repository	Métodos além do CRUD Panache
UserRepository	findByEmail , listAllOrdered
CriseRepository	listAllOrdered , countByStatus , countByNivel
AcaoCriseRepository	listByCrise (por crise, mais recente primeiro)
RelatorioCriseRepository	listAllOrdered , existsByCrise
AuditoriaRepository	search , distinctEntidades/Acoes/Usuarios
Departamento / TipoCrise / Instituicao / Campus	listAllOrdered (por nome)
CenarioCrise	listAllOrdered (por descrição)

7. Camada de serviços (BO) — regras de negócio

Todos `@ApplicationScoped` ; métodos de escrita são `@Transactional` . Uma técnica recorrente: como as entidades são *managed* dentro da transação, **alterar um campo já persiste** — não é preciso chamar `persist` no `update` .

7.1 CriseService — o coração do domínio

Criar: a crise sempre nasce `ABERTA` e ativa, independentemente do que o cliente enviar:

```
@Transactional
public CriseResponse create(CriseRequest request) {
    Crise crise = new Crise();
    crise.titulo = request.titulo(); crise.descricao = request.descricao();
    crise.nivel = request.nivel(); crise.tipoCriseId = request.tipoCriseId();
    crise.campusId = request.campusId(); crise.cenarioId = request.cenarioId();
    crise.departamentoId = request.departamentoId();
    crise.status = StatusCrise.ABERTA; // forçado pelo servidor
    crise.active = true;
    crise.createdAt = LocalDateTime.now();
    criseRepository.persist(crise);
    return CriseResponse.from(crise);
}
```

Mudar status: uma crise `ENCERRADA` é terminal — não muda mais:

```
@Transactional
public CriseResponse changeStatus(Long id, StatusCrise novoStatus) {
    Crise crise = getOrThrow(id);
    if (crise.status == StatusCrise.ENCERRADA)
        throw new BadRequestException("Não é possível alterar o status de uma crise encerrada.");
    crise.status = novoStatus; // persistido pela transação
    return CriseResponse.from(crise);
}
```

Registrar ação: a regra de negócio mais importante — o usuário `COMUM` só atua no próprio departamento; o autor sai do JWT:

```
@Transactional
public AcaoCriseResponse addAcao(Long criseId, AcaoCriseRequest request) {
    Crise crise = getOrThrow(criseId);
    User autor = usuarioAtual(); // resolvido do subject do JWT
    if (autor.perfil == PerfilUsuario.COMUM
        && !crise.departamentoId.equals(autor.departamentoId))
        throw new ForbiddenException("Você só pode registrar ações em crises do seu departamento.");
    AcaoCrise acao = new AcaoCrise();
    acao.descricao = request.descricao(); acao.tipo = request.tipo();
    acao.criseId = crise.id; acao.usuarioId = autor.id;
    acao.createdAt = LocalDateTime.now();
    acaoCriseRepository.persist(acao);
    return AcaoCriseResponse.from(acao);
}

private User usuarioAtual() {
    Long id = Long.valueOf(jsonWebToken.getSubject());
    return userRepository.findByIdOptional(id).orElseThrow( ... );
}
```

7.2 RelatorioCriseService — relatório único e tardio

```
@Transactional
public RelatorioCriseResponse create(RelatorioCriseRequest request) {
    Crise crise = criseRepository.findByIdOptional(request.criseId()).orElseThrow( ... );
    if (crise.status != StatusCrise.RESOLVIDA && crise.status != StatusCrise.ENCERRADA)
        throw new BadRequestException("Só é possível gerar relatório de crises RESOLVIDAS ou ENCERRADAS.");
    if (relatorioCriseRepository.existsByCrise(crise.id))
        throw new BadRequestException("Esta crise já possui um relatório.");
    // ... preenche campos ...
    relatorio.usuarioId = Long.valueOf(jsonWebToken.getSubject());
    relatorioCriseRepository.persist(relatorio);
    return RelatorioCriseResponse.from(relatorio);
}
```

Dupla proteção contra duplicidade: checagem `existsByCrise` + constraint `UNIQUE` em `crise_id` no banco.

7.3 UsuarioService — CRUD administrativo com salvaguardas

- **E-mail único** na criação e na edição (excluindo o próprio registro).
- **Senha**: obrigatória ao criar; na edição, vazia/nula mantém a atual, preenchida gera novo hash BCrypt.
- **Auto-proteção**: não é possível desativar o próprio usuário.

```
@Transactional
public void deactivate(Long id) {
    User user = getOrThrow(id);
    Long atual = Long.valueOf(jsonWebToken.getSubject());
    if (user.id.equals(atual))
        throw new BadRequestException("Você não pode desativar o próprio usuário.");
    user.active = false; // soft-delete
}
```

7.4 DashboardService — agregação

```
public DashboardResponse resumo() {
    return new DashboardResponse(
        criseRepository.count(),
        criseRepository.countByStatus(StatusCrise.ABERTA),
        criseRepository.countByStatus(StatusCrise.EM_ANDAMENTO),
        criseRepository.countByStatus(StatusCrise.RESOLVIDA),
        criseRepository.countByNivel(NivelCriticidade.CRITICO));
}
```

7.5 Serviços de referência (Campus, Departamento, Instituicao, TipoCrise, CenarioCrise)

Compartilham a mesma forma: `listAll`, `findById`, `create` (define `active=true`), `update` (muta a entidade managed) e `deactivate` (soft-delete). Exemplo do `CampusService.update` — sem `persist` :

```
@Transactional
public CampusResponse update(Long id, CampusRequest request) {
    Campus campus = getOrThrow(id); // entidade gerenciada
    campus.nome = request.nome();
    campus.instituicaoId = request.instituicaoId();
    return CampusResponse.from(campus); // flush automático no fim da transação
}
```

8. Camada de controllers (JAX-RS)

Fino por design: recebem HTTP, validam com `@Valid`, aplicam segurança e delegam ao service. A anotação de classe define a política padrão; métodos podem endurecê-la.

PADRÃO — CRISECONTROLLER

```
@Path("/api/crises") @Authenticated
@Consumes(APPLICATION_JSON) @Produces(APPLICATION_JSON)
public class CriseController {
    @Inject CriseService criseService;

    @GET public List<CriseResponse> listAll() { return criseService.listAll(); }

    @POST @RolesAllowed("ADMIN")
    public Response create(@Valid CriseRequest request) {
        return Response.status(CREATED).entity(criseService.create(request)).build();
    }

    @PATCH @Path("/{id}/status") @RolesAllowed("ADMIN")
    public CriseResponse changeStatus(@PathParam("id") Long id, @Valid StatusUpdateRequest r) { ... }

    @POST @Path("/{id}/acoes") @RolesAllowed({"ADMIN", "COMUM"})
    public Response addAcao( ... ) { ... }
}
```

POLÍTICA DE ACESSO — RESUMO

Padrão do sistema: **leituras liberadas a qualquer autenticado; escritas só ADMIN**. Exceção: registrar ação de crise (ADMIN+COMUM). Usuários e auditoria são inteiramente ADMIN.

MAPA COMPLETO DE ENDPOINTS

Método · Rota	Acesso	Função
POST /auth/login	PÚBLICO	Autentica, seta cookie jwt
POST /auth/logout	AUTH	Expira o cookie
GET /auth/me	AUTH	Usuário da sessão
GET /api/dashboard/resumo	AUTH	Indicadores
GET /api/crises · /{id}	AUTH	Lista / detalha
POST /api/crises	ADMIN	Cria (nasce ABERTA)
PATCH /api/crises/{id}/status	ADMIN	Avança status
GET /api/crises/{id}/acoes	AUTH	Lista ações
POST /api/crises/{id}/acoes	ADMIN+COMUM	Registra ação
GET /api/relatorios · /{id}	AUTH	Lista / detalha
POST /api/relatorios	ADMIN	Cria relatório
GET/POST/PUT/DELETE /api/usuarios	ADMIN	CRUD de usuários
GET /api/auditoria · /facets	ADMIN	Logs + filtros
GET /api/{ref} · /{id}	AUTH	Lista / detalha referência
POST/PUT/DELETE /api/{ref}	ADMIN	CRUD referência (soft-delete)

{ref} ∈ departamentos · tipos-crise · instituicoes · campi · cenarios

9. DTOs e validação (Bean Validation)

Entrada e saída são `record`s. Os `*Request` declaram as regras de validação, disparadas pelo `@Valid` no controller; violações viram `400` automaticamente.

```
public record CriseRequest(
    @NotBlank(message="O título é obrigatório") @Size(max=200) String titulo,
    @NotBlank @Size(max=1000) String descricao,
    @NotNull NivelCriticidade nivel,
    @NotNull Long tipoCriseId,
    @NotNull Long campusId,
    Long cenarioId,           // opcional (sem @NotNull)
    @NotNull Long departamentoId) {}
```

Os `*Response` convertem a entidade via um método fábrica estático, garantindo que a entidade JPA nunca é serializada diretamente (e a senha jamais sai):

```
public record CriseResponse(Long id, String titulo, ..., LocalDateTime createdAt) {
    public static CriseResponse from(Crise c) {
        return new CriseResponse(c.id, c.titulo, ..., c.createdAt);
    }
}
```

UsuarioRequest ilustra validações compostas: `@Email`, `@Size(min=6,max=100)` na senha (opcional na edição) e `@NotNull` no perfil. O **UsuarioResponse** expõe id, name, email, perfil, departamentoid, active, createdAt — **nunca** a senha.

10. Segurança — JWT em cookie, passo a passo

O token não trafega em `Authorization: Bearer` ; ele é lido de um **cookie HttpOnly** chamado `jwt` . Como o JavaScript não acessa cookies HttpOnly, o token fica protegido contra roubo por XSS.

Emissão do token — `AuthService.login`

```
public String login(String email, String password) {
    User user = userRepository.findByEmail(email).orElseThrow(InvalidCredentialsException::new);
    if (!user.active || !BcryptUtil.matches(password, user.password))
        throw new InvalidCredentialsException();
    return Jwt.issuer(issuer)
        .subject(user.id.toString())           // id → "quem sou"
        .claim("name", user.name)
        .claim("email", user.email)
        .groups(user.perfil.name())           // "ADMIN"/"COMUM" → role
        .expiresIn(TOKEN_EXPIRATION_SECONDS) // 3600s = 1 hora
        .sign();                             // assina com privateKey.pem (RS256)
}
```

Entrega no cookie — `AuthController` + `AuthCookieFactory`

```
// AuthController.login
return Response.ok(new LoginResponse(true, "Login realizado com sucesso"))
    .cookie(authCookieFactory.buildLoginCookie(token, TOKEN_EXPIRATION_SECONDS)).build();

// AuthCookieFactory - atributos de segurança do cookie
new NewCookie.Builder("jwt")
    .value(token).path("/")
    .httpOnly(true)           // inacessível ao JS
    .sameSite(NewCookie.SameSite.LAX) // mitiga CSRF
    .maxAge((int) maxAgeSeconds).build();
```

Fluxo completo

- 1 Cliente faz `POST /auth/login` (rota `@PermitAll`).
- 2 Servidor valida BCrypt e assina o JWT.
- 3 Devolve o token no cookie `jwt` HttpOnly.
- 4 Browser reenvia o cookie automaticamente a cada requisição.
- 5 SmallRye extrai/verifica o token do cookie; `@Authenticated` / `@RolesAllowed` autorizam.
- 6 `POST /auth/logout` devolve o cookie com `maxAge=0` , expirando-o.

Camadas de autorização

Mecanismo	Onde	Efeito
<code>@PermitAll</code>	login	Acesso anônimo
<code>@Authenticated</code>	reads, /me, dashboard	Qualquer perfil logado
<code>@RolesAllowed("ADMIN")</code>	escritas, usuários, auditoria	Só administrador (claim <code>groups</code>)
<code>@RolesAllowed({"ADMIN", "COMUM"})</code>	registrar ação	Ambos os perfis
Regra no BO	<code>CriseService.addAcao</code>	COMUM restrito ao próprio departamento (403)
Regra no BO	<code>UsuarioService.deactivate</code>	Bloqueia auto-desativação (400)

11. Auditoria automática por filtro de resposta

Toda ação de modificação é registrada de forma transparente por um `ContainerResponseFilter` — os services não precisam chamar auditoria manualmente. Ele roda na *resposta* para capturar o status HTTP.

```
@Provider
public class AuditoriaLogFilter implements ContainerResponseFilter {
    @Override
    public void filter(ContainerRequestContext req, ContainerResponseContext resp) {
        if (!isModificacao(req.getMethod())) return; // só POST/PUT/PATCH/DELETE
        String subject = jwt.getSubject();
        if (subject == null || subject.isBlank()) return; // login anônimo não é auditado
        String entidade = extrairEntidade(normalize(req.getUriInfo().getPath()));
        if ("auth".equals(entidade) || "auditoria".equals(entidade)) return; // evita ruído
        try {
            auditoriaService.registrar(Long.valueOf(subject), nomeDoToken(subject),
                acaoDe(req.getMethod()), req.getMethod(), entidade,
                "/" + path, resp.getStatus());
        } catch (RuntimeException e) {
            LOG.warnf(e, "Falha ao registrar auditoria ... "); // nunca quebra a requisição
        }
    }
    // POST→CRIAR, PUT/PATCH→ATUALIZAR, DELETE→EXCLUIR
}
```

Transação própria: o `AuditoriaService.registrar` é `@Transactional(REQUIRES_NEW)`, porque o filtro executa *depois* que a transação da requisição original já foi encerrada. Assim o log é gravado numa transação independente e uma falha de auditoria não desfaz (nem depende de) a operação principal.

O que fica registrado em cada linha de `auditoria_logs`: quem (id + nome do token), ação amigável, método HTTP, entidade derivada do path, recurso completo, status HTTP e data/hora. Índices em `data_hora` DESC e `entidade_afetada` aceleram as consultas com filtro.

12. Tratamento de exceções

Situação	Exceção	HTTP
Login inválido (e-mail, inativo, senha)	<code>InvalidCredentialsException</code> + <code>Mapper</code>	401 com JSON <code>LoginResponse(false, msg)</code>
Entidade inexistente	<code>NotFoundException</code>	404
Regra violada (crise encerrada, e-mail duplicado, relatório duplicado, auto-desativação, status inválido)	<code>BadRequestException</code>	400
COMUM fora do departamento	<code>ForbiddenException</code>	403
Falha de validação de DTO	Bean Validation	400 automático

```
@Provider
public class InvalidCredentialsExceptionMapper
    implements ExceptionMapper<InvalidCredentialsException> {
    @Override public Response toResponse(InvalidCredentialsException ex) {
        return Response.status(UNAUTHORIZED).type(APPLICATION_JSON)
            .entity(new LoginResponse(false, ex.getMessage())).build();
    }
}
```

As demais exceções (`NotFound` , `BadRequest` , `Forbidden`) usam os mappers embutidos do Quarkus/RESTEasy — só o caso de credenciais tem mapper próprio, para devolver um corpo JSON amigável.

13. Migrações Flyway (V1–V8)

O schema evolui de forma versionada e reproduzível. As migrações rodam em ordem no start.

Versão	Conteúdo
V1	users (email UNIQUE). Ainda sem perfil/departamento.
V2	departamentos e tipos_crise + seeds ("Coordenação de TI"/CTI e "Tecnologia").
V3	instituicoes , campi (FK→instituicoes), cenarios_crise (FK→tipos_crise, departamentos).
V4	crises com FKs (cenario_id nullable).
V5	acoes_crise (FK→crises, users).
V6	relatorios_crise — crise_id UNIQUE (1 relatório por crise).
V7	Adiciona perfil (default 'COMUM') e departamento_id em users; promove o admin a ADMIN.
V8	auditoria_logs + índices data_hora DESC e entidade_afetada .

```
-- V6: garante 1 relatório por crise já no schema
CREATE TABLE relatorios_crise (
  id          BIGSERIAL      PRIMARY KEY,
  ...
  crise_id    BIGINT  NOT NULL UNIQUE REFERENCES crises (id),
  usuario_id  BIGINT  NOT NULL REFERENCES users (id),
  created_at  TIMESTAMP NOT NULL DEFAULT now()
);
```

14. Ciclo de vida completo de uma crise (walkthrough)

Juntando todas as camadas, veja o caminho de ponta a ponta:

- 1 LOGIN**
POST /auth/login → AuthService valida BCrypt → JWT no cookie jwt
- 2 ABERTURA (ADMIN)**
POST /api/crises → @RolesAllowed(ADMIN) → CriseService.create()
status = ABERTA (forçado) ; AuditoriaLogFilter grava CRIAR/crises/201
- 3 ATUAÇÃO (ADMIN ou COMUM)**
POST /api/crises/{id}/acoes → regra: COMUM só no próprio depto.
nova AcaoCrise com autor = subject do JWT ; auditado
- 4 EVOLUÇÃO DO STATUS (ADMIN)**
PATCH /api/crises/{id}/status
ABERTA → EM_ANDAMENTO → RESOLVIDA → ENCERRADA
(ENCERRADA é terminal: bloqueia novas mudanças → 400)
- 5 RELATÓRIO FINAL (ADMIN)**
POST /api/relatorios → exige RESOLVIDA/ENCERRADA e 1 por crise
- 6 RASTREABILIDADE**

Cada POST/PUT/PATCH/DELETE gerou 1 linha em auditoria_logs
consultável em /api/auditoria (só ADMIN)